



Manara AWS Solutions Architect Associate Learning Program

AWS Solutions Architect - Associate

Deployment of a Containerized Microservices Architecture on AWS

AWS Solutions Architect - Associate Graduation Project

Migration of a Node.js application to an architecture composed of three containerized microservices, deployed on AWS ECS Fargate and integrated with private networking, an Application Load Balancer, AWS Cloud Map, ElastiCache Redis, Secrets Manager, CodePipeline, CodeBuild, ECR, S3, and OpenTelemetry/X-Ray observability.

Prepared by: Salmane Baba

Email: salmanebaba@outlook.com

LinkedIn: [linkedin.com/in/salmanebaba](https://www.linkedin.com/in/salmanebaba)

Project repository: [Containerized-Microservices-with-ECS-Fargate-and-Service-Discovery](#)

September 2026

Contents

1	Introduction and Objectives	1
1.1	Project Background	1
1.2	Context	1
1.3	Problem Statement	1
1.4	Technical Objectives	2
2	Application Architecture	3
2.1	Functional Decomposition	3
2.2	Main Order Flow	3
2.3	Local Execution with Docker Compose	4
3	AWS Architecture	5
3.1	Overall View	5
3.2	Region and VPC	6
3.3	Internet Gateway and NAT Gateway	6
3.4	Application Load Balancer	7
4	Deployment on Amazon ECS Fargate	8
4.1	Why Fargate	8
4.2	Task Definitions	8
4.3	Environment Variables and Service Discovery	9
4.4	ECR and Image Versioning	9
5	Security, Secrets, and Redis	11
5.1	Security Groups	11
5.2	AWS Secrets Manager	11
5.3	ElastiCache for Redis	12
5.4	IAM: Role Separation	12
6	CI/CD Pipeline	13
6.1	General Flow	13
6.2	CodeBuild and buildspec	13
6.3	S3 Artifact Store	14
6.4	CodeDeploy: Project Status	14
7	Testing and Validation Interface	15
7.1	Tests with curl	15
7.2	Test Frontend	16
8	Observability: CloudWatch, OpenTelemetry, and X-Ray	18
8.1	CloudWatch Logs	18
8.2	Why Distributed Tracing?	18

8.3	OpenTelemetry Instrumentation	18
8.4	ADOT Collector Sidecar	19
9	Infrastructure as Code with CloudFormation IaC Generator	20
9.1	Template Generation	20
9.2	Limitations of the Generated Template	21
9.3	Required Cleanup	21
10	Validation and Results	23
11	Challenges and Technical Decisions	24
11.1	The <code>latest</code> Tag and Image Versioning	24
11.2	ALB Routing and Application Paths	24
11.3	Local Nginx Configuration	24
11.4	CodeDeploy Unavailable	24
12	Future Improvements	25
13	Conclusion	26
A	Useful Commands	27
A.1	Local Execution	27
A.2	Stopping Local Services	27
A.3	Nginx Tests	27
A.4	Generating Traffic for X-Ray	27
B	X-Ray IAM Policy	28

List of Figures

3.1	Overall AWS architecture of the project	5
3.2	VPC, Availability Zones, and the four subnets	6
3.3	HTTP:80 listener rules of the Application Load Balancer	7
4.1	ECS cluster with the three services in 1/1 running state	8
4.2	AWS Cloud Map namespace <code>microservices.local</code> and registered services	9
4.3	ECR repositories and image tags generated by CodeBuild	10
5.1	JWT secret in AWS Secrets Manager	11
5.2	ElastiCache Redis configuration and private subnet group	12
6.1	Successful pipeline execution from GitHub to CodeBuild	13
6.2	Build artifact stored in the CodePipeline S3 bucket	14
7.1	Health checks	15
7.2	Login request	15
7.3	Order request	16
7.4	API test frontend	17
8.1	X-Ray / CloudWatch Trace Map after generating traffic	19
9.1	CloudFormation IaC Generator after scanning resources	21

List of Tables

2.1	Application microservices	3
3.1	Network addressing plan	6
3.2	ALB routing rules	7
4.1	Main ECS task configuration	8

Chapter 1

Introduction and Objectives

1.1 Project Background

This project was developed as part of the **Manara AWS Solutions Architect Associate learning program**. Within the program, learners were invited to complete a graduation project demonstrating their ability to design and implement a complete AWS architecture, document the solution, and present it through an architecture diagram and a public GitHub repository.

The selected topic was “**Containerized Microservices with ECS Fargate and Service Discovery**”. The original project objective proposed migrating a monolithic Node.js application into three microservices—*Auth*, *Orders*, and *Notifications*—deployed on Amazon ECS Fargate. The suggested architecture included Amazon ECR, Application Load Balancer path-based routing, AWS Cloud Map for service discovery, AWS Secrets Manager, Amazon ElastiCache for Redis, CI/CD with CodePipeline and CodeDeploy, and AWS X-Ray for distributed tracing.

This implementation was used as an opportunity to go beyond the basic deployment requirements by configuring the networking architecture, private and public subnets, IAM roles, containerized CI/CD workflows, centralized secret management, observability with OpenTelemetry and AWS X-Ray, and infrastructure export through AWS CloudFormation IaC Generator.

1.2 Context

The objective of this project is to build a modern cloud architecture from a Node.js application split into several independent services. The work goes beyond container deployment: it also covers network design, security, service discovery, secret management, Redis caching, the CI/CD pipeline, observability, and an initial step toward Infrastructure as Code.

The project is based on three microservices:

- **Auth Service:** authentication, JWT generation and validation;
- **Order Service:** order creation and retrieval;
- **Notification Service:** creation and storage of notifications triggered by application events.

1.3 Problem Statement

A monolithic architecture can quickly become difficult to evolve as the number of features, teams, or deployments grows. Splitting an application into microservices helps isolate responsibilities, but it introduces new challenges: service-to-service communication, network discovery, security, resilience, configuration management, monitoring, and deployment automation.

The project therefore addresses the following question:

How can a containerized microservices architecture be designed, deployed, and automated on AWS while keeping the network secure, the services stateless, service discovery dynamic, and the CI/CD pipeline reproducible?

1.4 Technical Objectives

The main objectives are:

1. Containerize the three services using Docker.
2. Deploy the containers on Amazon ECS Fargate without managing EC2 servers for the workloads.
3. Isolate application tasks in private subnets.
4. Expose the APIs through an Application Load Balancer using path-based routing.
5. Configure AWS Cloud Map for DNS-based service discovery between microservices.
6. Store the JWT secret in AWS Secrets Manager.
7. Use ElastiCache for Redis over TLS to prepare shared state/cache management.
8. Automate image build and push operations with CodePipeline and CodeBuild.
9. Version Docker images with the build number instead of relying only on `latest`.
10. Centralize logs and prepare distributed tracing with OpenTelemetry, ADOT, and AWS X-Ray.
11. Generate an initial CloudFormation model from existing resources to begin an Infrastructure as Code approach.

Chapter 2

Application Architecture

2.1 Functional Decomposition

The application decomposition follows the *single responsibility* principle. Each service exposes its API on a dedicated port and can evolve independently.

Table 2.1: Application microservices

Service	Port	Responsibility
Auth Service	4001	Authentication, JWT issuance, token validation, and Redis connectivity.
Order Service	4002	Order creation/retrieval, token validation through the Auth service, then invocation of the Notification service.
Notification Service	4003	Event reception, notification generation, and notification retrieval.

2.2 Main Order Flow

The most representative scenario is order creation:

1. The client authenticates with the Auth service and receives a JWT.
2. The client sends an order creation request to the Order service.
3. Order contacts Auth through the private network to validate the token.
4. If the token is valid, Order creates the order.
5. Order calls Notification to create an event notification.
6. The aggregated response is returned to the client.

This flow justifies the use of service discovery and, later, distributed tracing to follow a request across several components.

2.3 Local Execution with Docker Compose

Before deployment to AWS, the services are tested locally using Docker Compose. Docker service names act as internal DNS names:

Listing 2.1: Simplified excerpt of the local Docker Compose configuration

```
1 services:
2   auth-service:
3     ports:
4       - "4001:4001"
5
6   order-service:
7     environment:
8       AUTH_SERVICE_URL: http://auth-service:4001
9       NOTIFICATION_SERVICE_URL: http://notification-service:4003
10    ports:
11      - "4002:4002"
12
13   notification-service:
14     ports:
15       - "4003:4003"
```

This step validates the service decomposition and service-to-service calls before introducing AWS-specific complexity.

Chapter 3

AWS Architecture

3.1 Overall View

The following figure presents the final architecture designed for the project. It separates regional managed services, the VPC network, public/private subnets, ECS tasks, and the CI/CD pipeline.

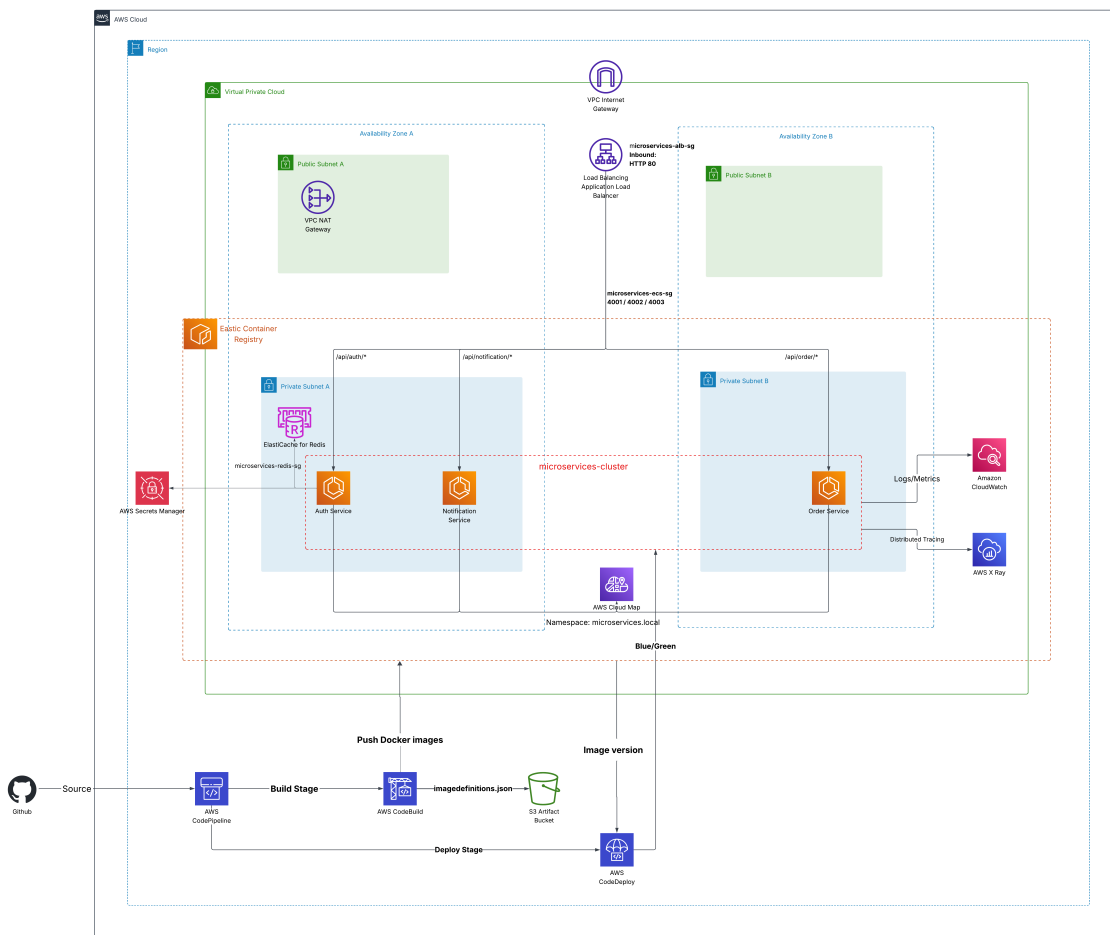


Figure 3.1: Overall AWS architecture of the project

3.2 Region and VPC

All main resources are deployed in the **eu-west-3** region (Europe – Paris). A dedicated VPC named **microservices-vpc** was created with the CIDR block **10.0.0.0/16**.

Table 3.1: Network addressing plan

Resource	CIDR	Role
VPC	10.0.0.0/16	Main network for the project
Public Subnet A	10.0.1.0/24	ALB / NAT Gateway, AZ A
Public Subnet B	10.0.2.0/24	ALB, AZ B
Private Subnet A	10.0.11.0/24	ECS tasks / Redis
Private Subnet B	10.0.12.0/24	ECS tasks / Redis

Using two Availability Zones avoids designing the architecture around a single failure domain. Fargate tasks run in private subnets, while the ALB is associated with both public subnets.

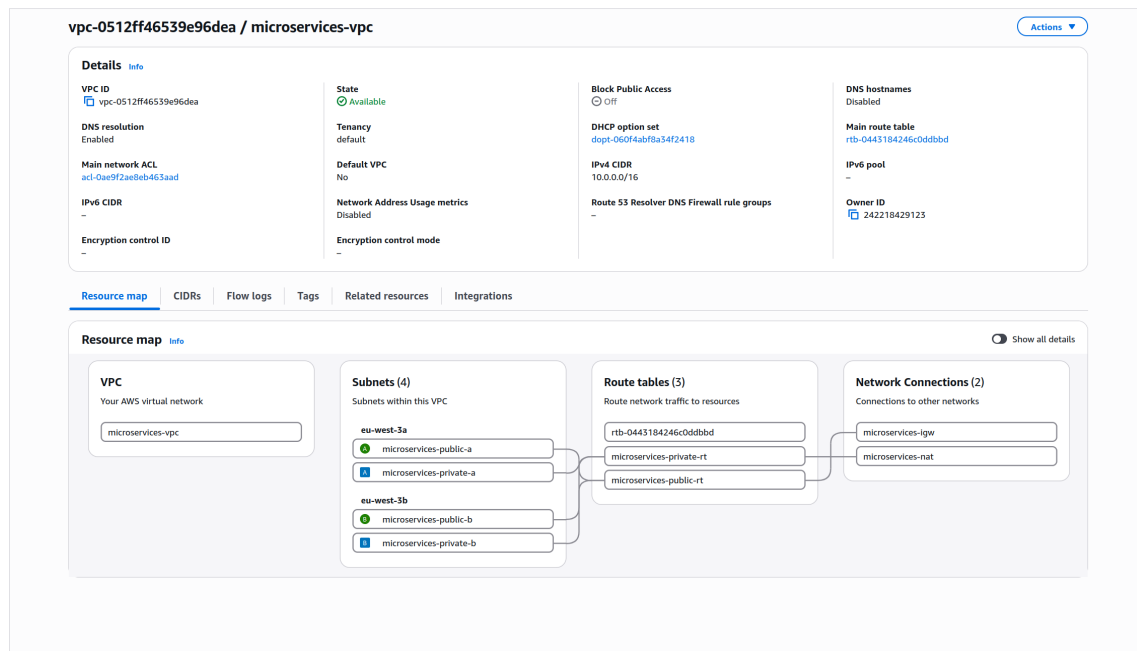


Figure 3.2: VPC, Availability Zones, and the four subnets

3.3 Internet Gateway and NAT Gateway

The Internet Gateway is attached to the VPC to provide Internet connectivity for public resources. ECS tasks remain private and use a NAT Gateway deployed in **microservices-public-a** for required outbound access, for example to retrieve images or contact public AWS endpoints.

Only one NAT Gateway was used to limit project cost. In a highly available production architecture, one NAT Gateway per Availability Zone would generally be preferred to avoid an inter-AZ dependency.

Note

Because the Fargate tasks are private, they do not receive public IPv4 addresses. They remain reachable from the ALB and from other services explicitly allowed by Security Groups.

3.4 Application Load Balancer

An internet-facing Application Load Balancer named `microservices-alb` receives HTTP traffic and applies path-based routing.

Table 3.2: ALB routing rules

Priority	Path pattern	Target Group
10	<code>/api/auth/*</code>	<code>microservices-auth-tg</code> on port 4001
20	<code>/api/notification/*</code>	<code>microservices-notification-tg</code> on port 4003
30	<code>/api/order/*</code>	<code>microservices-order-tg</code> on port 4002
Default	Other paths	Fixed HTTP 404 response

The Target Groups use target type `ip`, which is appropriate for Fargate tasks using `awsvpc` network mode. Health checks are performed on:

- `/api/auth/health`
- `/api/order/health`
- `/api/notification/health`

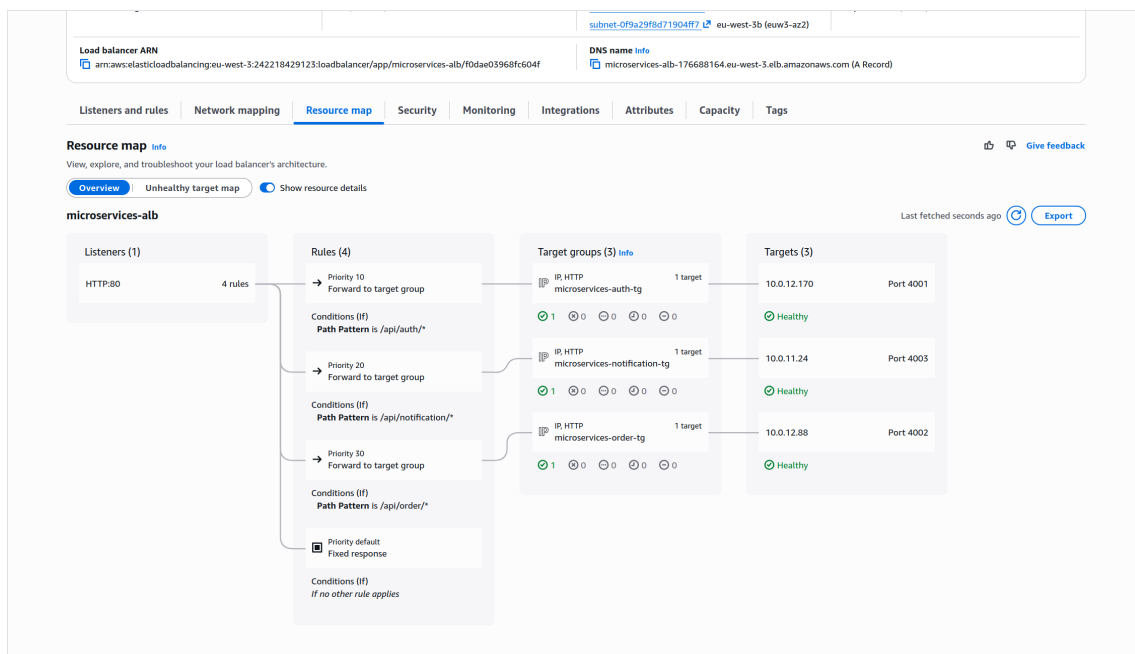


Figure 3.3: HTTP:80 listener rules of the Application Load Balancer

Chapter 4

Deployment on Amazon ECS Fargate

4.1 Why Fargate

Amazon ECS Fargate runs containers without requiring the provisioning or administration of the underlying EC2 instances. This simplifies the project and keeps the focus on task definitions, security, networking, and automation.

The cluster used is `microservices-cluster`. Tasks use `awsvpc` network mode, which assigns a network interface to each task and allows fine-grained control through Security Groups.

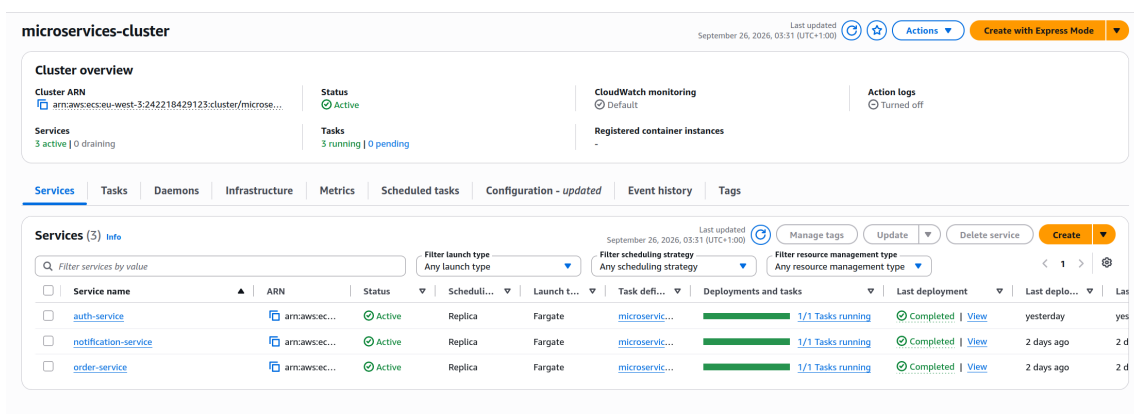


Figure 4.1: ECS cluster with the three services in 1/1 running state

4.2 Task Definitions

Each microservice has its own Task Definition. The main settings are summarized below.

Table 4.1: Main ECS task configuration

Service	CPU	Memory	Port	ECR image
Auth	256 CPU	1024 MiB	4001	containerized-microservices-auth:<build>
Order	256 CPU	512 MiB	4002	containerized-microservices-order:<build>
Notification	256 CPU	512 MiB	4003	containerized-microservices-notification:<build>

The Auth task has more memory because it also contains an OpenTelemetry Collector sidecar during tracing setup.

4.3 Environment Variables and Service Discovery

The Order service needs the internal endpoints of the Auth and Notification services. AWS Cloud Map provides a private DNS namespace:

Listing 4.1: Main environment variables of the Order service

```
1 AUTH_SERVICE_URL=http://auth-service.microservices.local:4001
2 NOTIFICATION_SERVICE_URL=http://notification-service.microservices.local:4003
3 PORT=4002
```

Using DNS names avoids hardcoding the ephemeral IP addresses of Fargate tasks. When a new task replaces an old one, ECS and Cloud Map update the service registration without requiring application code changes.

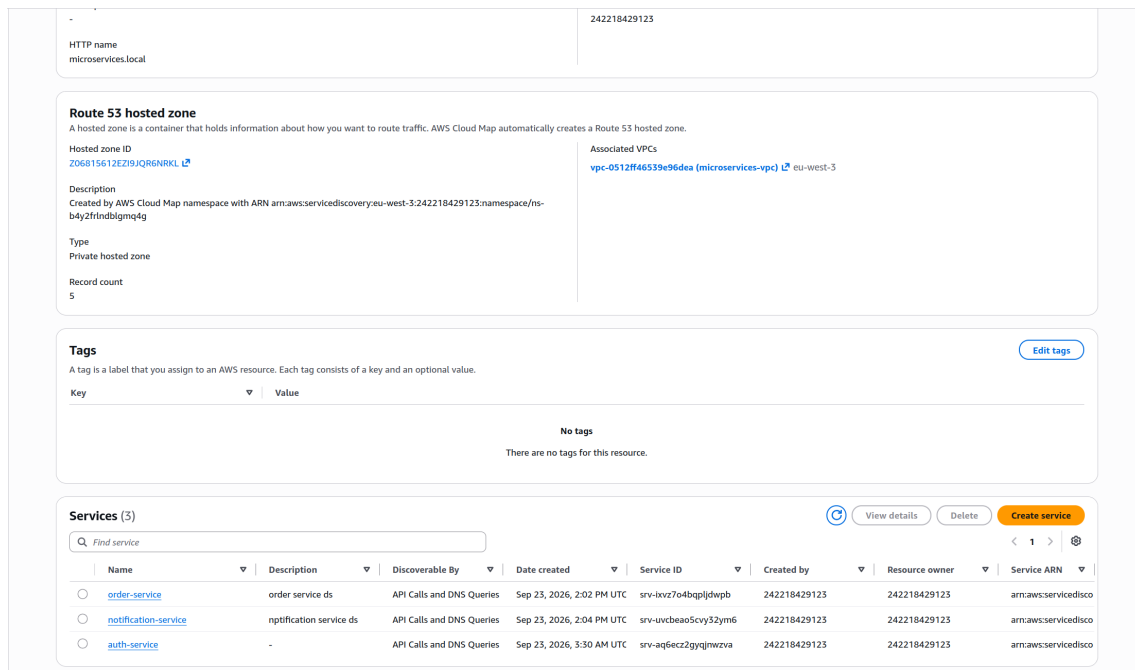


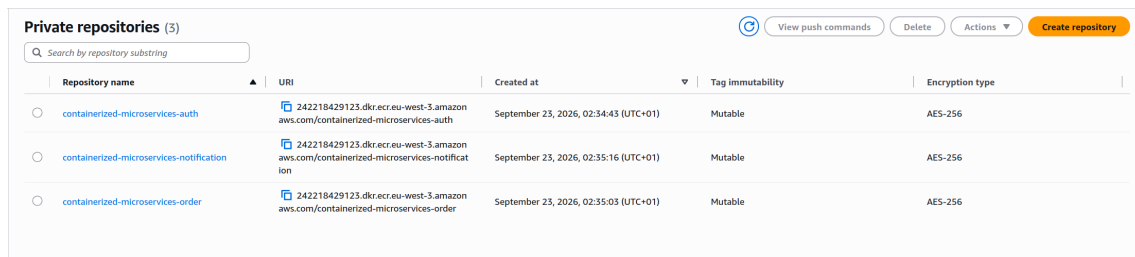
Figure 4.2: AWS Cloud Map namespace `microservices.local` and registered services

4.4 ECR and Image Versioning

Three private Amazon ECR repositories store the images:

- `containerized-microservices-auth`
- `containerized-microservices-order`
- `containerized-microservices-notification`

Images do not rely only on the `latest` tag. The pipeline uses the CodeBuild build number to produce traceable versions such as `:7`, `:8`, and so on. This approach makes it easier to identify an exact version and prepares the environment for rollback mechanisms.



Private repositories (3)

Search by repository substring

View push commands Delete Actions Create repository

	Repository name ▲	URI	Created at ▼	Tag immutability	Encryption type
☐	containerized-microservices-auth	242218429123.dkr.ecr.eu-west-3.amazonaws.com/containerized-microservices-auth	September 23, 2026, 02:34:43 (UTC+01)	Mutable	AES-256
☐	containerized-microservices-notification	242218429123.dkr.ecr.eu-west-3.amazonaws.com/containerized-microservices-notification	September 23, 2026, 02:35:16 (UTC+01)	Mutable	AES-256
☐	containerized-microservices-order	242218429123.dkr.ecr.eu-west-3.amazonaws.com/containerized-microservices-order	September 23, 2026, 02:35:03 (UTC+01)	Mutable	AES-256

Figure 4.3: ECR repositories and image tags generated by CodeBuild

Chapter 5

Security, Secrets, and Redis

5.1 Security Groups

Two main categories of network rules are used:

- **ALB Security Group:** allows HTTP/80 from the Internet;
- **ECS Security Group:** allows application ports 4001, 4002, and 4003 from the ALB and, when necessary, between ECS tasks.

Application ports should not be exposed directly to 0.0.0.0/0. External clients communicate only with the ALB.

5.2 AWS Secrets Manager

The Auth service JWT secret should not be stored in the Docker image or the Git repository. A secret named `microservices/auth/jwt` was therefore created in AWS Secrets Manager.

The intended configuration is to inject its value into the container through the Task Definition `Secrets` field:

Listing 5.1: Recommended form for injecting the JWT secret into ECS

```
1 Secrets:
2   - Name: JWT_SECRET
3     ValueFrom: <secret-ARN>
```

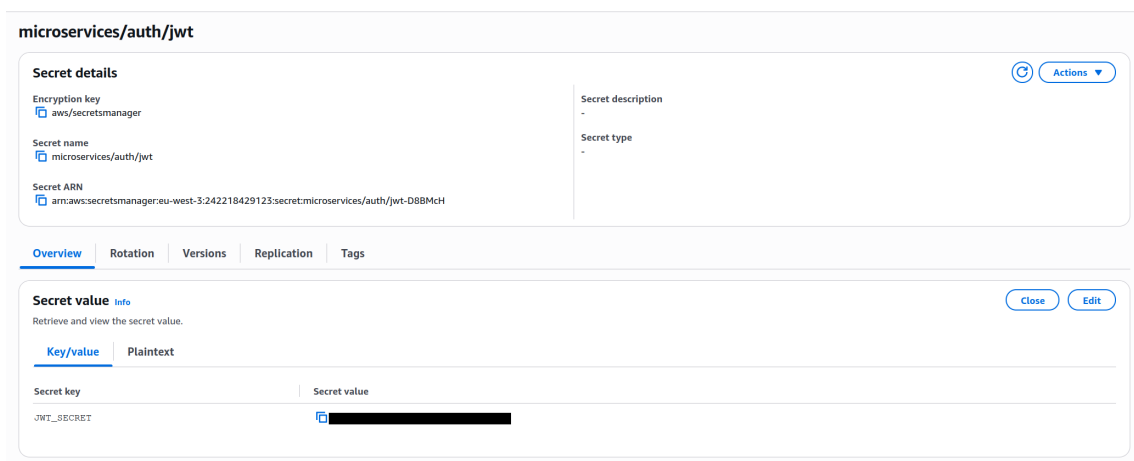


Figure 5.1: JWT secret in AWS Secrets Manager

5.3 ElastiCache for Redis

A Redis cluster is placed in the private subnets through the subnet group `microservices-redis-subnet-group`. The Auth service receives the endpoint through the `REDIS_URL` variable and uses a TLS URL such as:

```
1 rediss://master.<redis-endpoint>.euw3.cache.amazonaws.com:6379
```

Using `rediss://` encrypts client-to-Redis communication. In the current prototype, Redis is mainly prepared as a shared cache/state store; a natural next step is to use it for sessions or other state that should not remain in the local memory of one task.

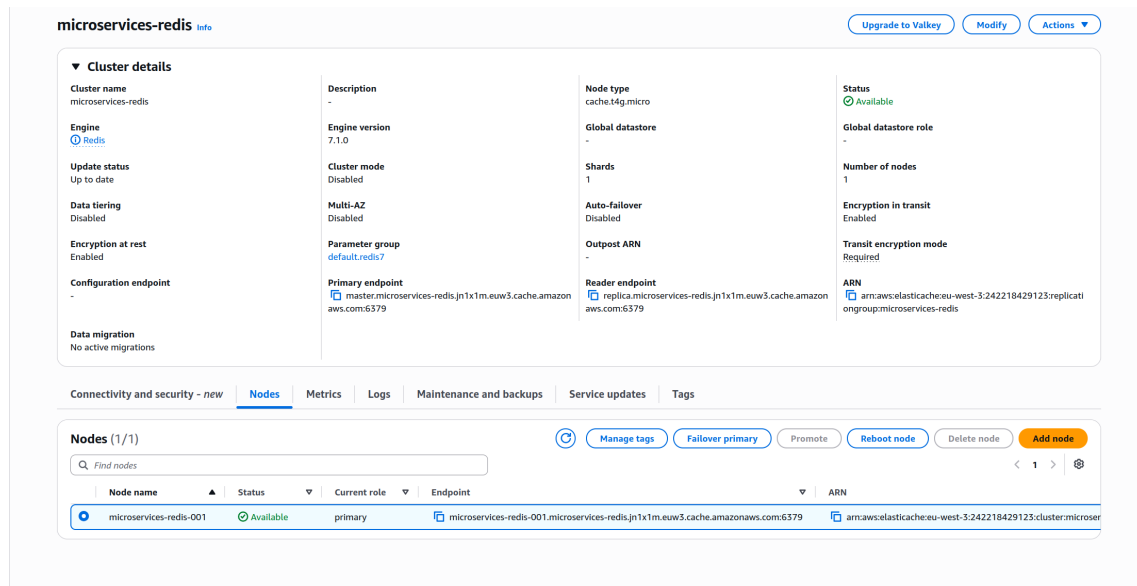


Figure 5.2: ElastiCache Redis configuration and private subnet group

5.4 IAM: Role Separation

Two roles are separated:

- **Task Execution Role:** used by ECS for platform operations such as pulling ECR images, sending logs to CloudWatch, and accessing startup secrets;
- **Task Role:** AWS permissions granted to the application code and sidecars running inside the task, including X-Ray permissions.

This separation follows the principle of least privilege and avoids granting application code the same permissions as the ECS runtime.

CI/CD Pipeline

The current pipeline automates the Source and Build stages:

The V2 pipeline watches the `main` branch. A push automatically triggers an execution. The source code is passed to CodeBuild as `SourceArtifact`. The build then produces Docker images and a `BuildArtifact` stored in S3.

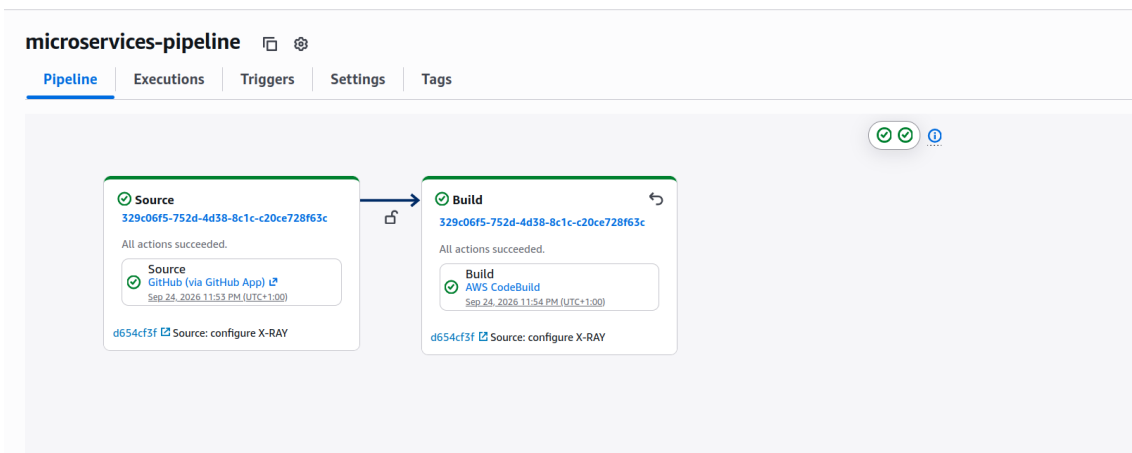


Figure 6.1: Successful pipeline execution from GitHub to CodeBuild

The CodeBuild project `microservices-build` uses the `buildspec.yml` file. The process is:

1. dynamically retrieve the AWS account ID;
2. authenticate to ECR;
3. build the three images;
4. tag each image with `CODEBUILD_BUILD_NUMBER`;
5. push the images;
6. generate `imagedefinitions.json` as a build artifact.

Listing 6.1: CodeBuild build process

```

1 AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
2
3 auth_image=$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/containerized-
  microservices-auth
4 order_image=$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/containerized-
  microservices-order
5 notification_image=$AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/containerized-
  microservices-notification
6
7 docker build -t $auth_image:$CODEBUILD_BUILD_NUMBER ./auth-service
8 docker build -t $order_image:$CODEBUILD_BUILD_NUMBER ./order-service
9 docker build -t $notification_image:$CODEBUILD_BUILD_NUMBER ./notification-service
10
11 docker push $auth_image:$CODEBUILD_BUILD_NUMBER
12 docker push $order_image:$CODEBUILD_BUILD_NUMBER
13 docker push $notification_image:$CODEBUILD_BUILD_NUMBER

```

6.3 S3 Artifact Store

The pipeline uses an S3 bucket as its Artifact Store. It does not store Docker images; those are stored in ECR. S3 stores pipeline artifacts, including the zipped source code and build outputs such as `imagedefinitions.json`.

Note

This separation is important: ECR is an OCI/Docker image registry, while S3 is used by CodePipeline to transfer and retain artifacts between stages.

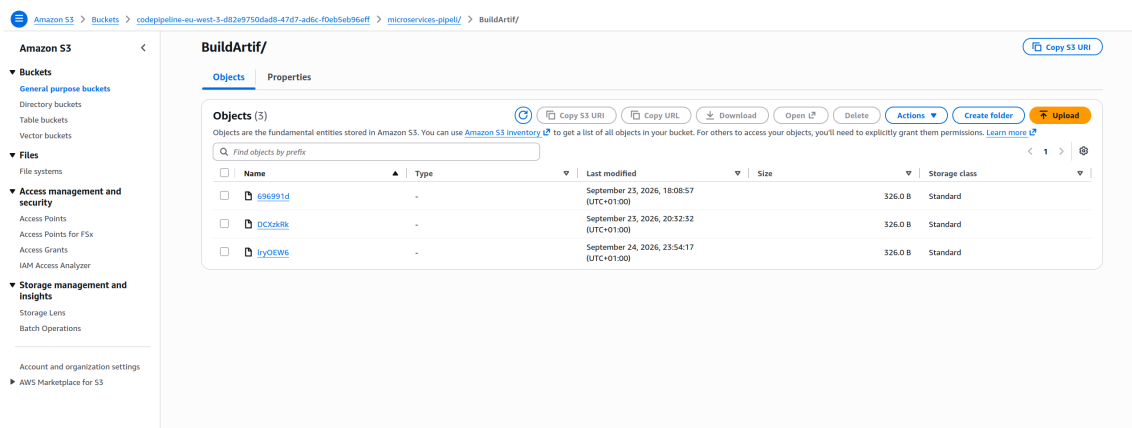


Figure 6.2: Build artifact stored in the CodePipeline S3 bucket

6.4 CodeDeploy: Project Status

The initial design planned an ECS Blue/Green deployment using AWS CodeDeploy. Green Target Groups were prepared, but access to CodeDeploy was blocked by an AWS account activation issue. This part is therefore considered **planned but not completed**.

Once available, the target evolution is:

CodePipeline → CodeBuild → ECR → CodeDeploy → ECS Blue/Green

Chapter 7

Testing and Validation Interface

7.1 Tests with curl

The APIs can be tested through the public DNS name of the ALB.

Listing 7.1: Health checks through the ALB

```
1 curl http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/auth/health
2 curl http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/order/health
3 curl http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/notification/health
```

```
blackstar@blackstar:~$ curl http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/auth/health
{"status":"ok","service":"auth-service"}blackstar@blackstar:~$ ^C
blackstar@blackstar:~$ curl http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/order/health
{"status":"ok","service":"order-service"}blackstar@blackstar:~$ ^C
blackstar@blackstar:~$ curl http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/notification/health
{"status":"ok","service":"notification-service"}blackstar@blackstar:~$ □
```

Figure 7.1: Health checks

Example login request:

```
1 curl -X POST \
2 http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/auth/login \
3 -H "Content-Type: application/json" \
4 -d '{"username":"alice","password":"password123"}'
```

```
blackstar@blackstar:~$ curl http://localhost:4001/health
{"status":"ok","service":"auth-service"}blackstar@blackstar:~$ ^C
blackstar@blackstar:~$ curl -X POST http://localhost:4001/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"password123"}'
{"message":"Login successful","token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1c2VybmlkLjIc2VybmFtZSI6ImFsaWVlLnFsaWVlIiwiaWF0IjoxNzkwMTI0ODUwLCJleHAiOiE3OTAxMjc0MTB9.kx_dZVl7PLz3MncR3KAiu06oi1h5aQ85gB04cShKKlo","user":{"id":"user-1","username":"alice"}}blackstar@blackstar:~$ □
```

Figure 7.2: Login request

Order creation can then be tested with the returned token:

```
1 curl -X POST \
2 http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/order/orders \
3 -H "Content-Type: application/json" \
4 -H "Authorization: Bearer <JWT>" \
5 -d '{"item":"Cloud Lab","total":99}'
```

```
blackstar@blackstar:~$ curl -X POST http://localhost:4002/orders \
-H "Content-Type: application/json" \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VybmlCJ1c2VybmFtZSI6ImFsaWNlIiwiaWF0Ijo
xNzkwMTI0NjY5LCJleHAiOiE3OTAxMjgyNjI9LzWjdZbnivvCCcbJhuO_x-GZ0L-TOug8R20MgHmNXnGs" \
-d '{"item":"Flight Ticket","total":250}'
{"message":"Order created successfully","order":{"id":"ORD-1790124846440","item":"Flight Ticket","total":250,"userId":"u
ser-1","username":"alice","status":"created"},"notification":{"message":"Notification stored successfully","notification
":{"id":"NOT-1790124846445","type":"order_created","message":"Order ORD-1790124846440 created for alice","order":{"id":"
ORD-1790124846440","item":"Flight Ticket","total":250,"userId":"user-1","username":"alice","status":"created"},"received
At":"2026-09-23T00:54:06.445Z"}}}blackstar@blackstar:~$
```

Figure 7.3: Order request

7.2 Test Frontend

A lightweight static frontend was created to test the main APIs without relying only on the command line. It can:

- check the health of all three services;
- test login and display the JWT;
- create/list orders;
- send/list notifications;
- display the HTTP status code, response time, and JSON body.

To avoid CORS issues, the frontend can be served locally with Nginx, while the `/api/` route is proxied to the AWS ALB.

Listing 7.2: Excerpt of the test Nginx configuration

```
1 server {
2     listen 80 default_server;
3     root /var/www/html;
4     index index.html;
5
6     location = / {
7         try_files /index.html =404;
8     }
9
10    location /api/ {
11        proxy_pass http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com;
12        proxy_http_version 1.1;
13        proxy_set_header Host microservices-alb-176688164.eu-west-3.elb.amazonaws.com;
14        proxy_set_header X-Real-IP $remote_addr;
15        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
16    }
17 }
```

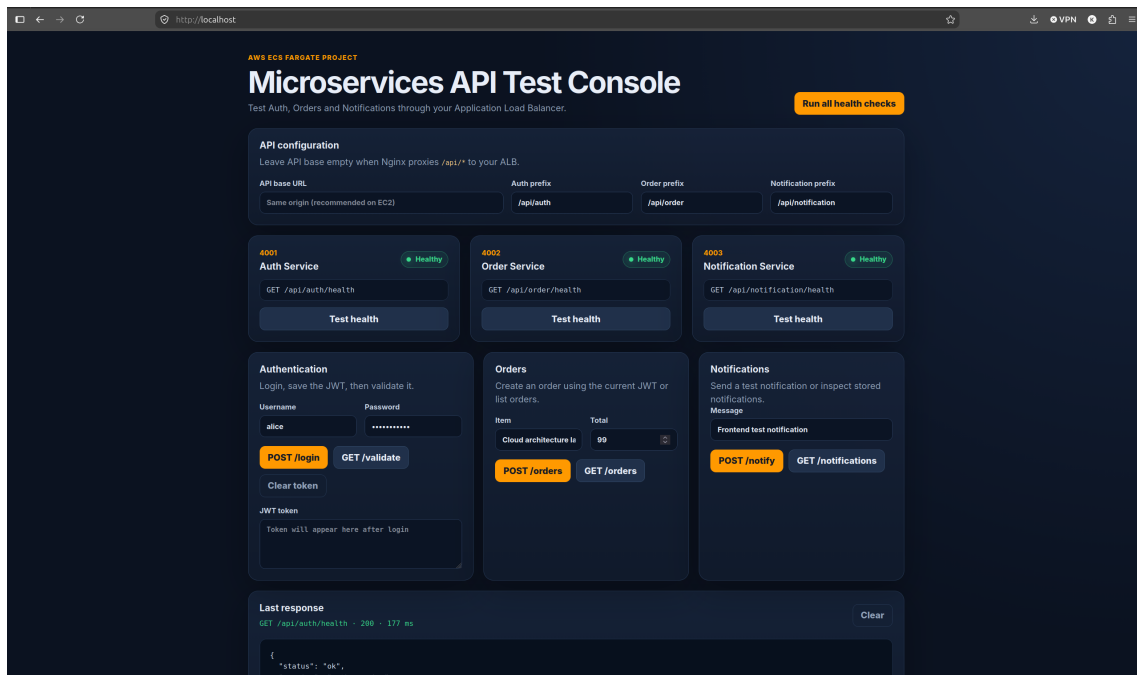


Figure 7.4: API test frontend

Chapter 8

Observability: CloudWatch, OpenTelemetry, and X-Ray

8.1 CloudWatch Logs

ECS containers use the `awslogs` log driver. Log groups such as `/ecs/microservices-auth` collect standard output from the services and the OpenTelemetry Collector.

This centralizes:

- startup messages;
- application errors;
- ADOT sidecar logs;
- events required to diagnose tracing issues.

8.2 Why Distributed Tracing?

With several microservices, a single client request can pass through Order, Auth, and Notification. Independent logs are not always enough to reconstruct the timeline. Distributed tracing adds a trace identifier and spans representing each stage.

The goal is to visualize, for example:

`Client → ALB → Order → Auth → Notification`

with latency and errors associated with each call.

8.3 OpenTelemetry Instrumentation

The Auth service was instrumented with OpenTelemetry for Node.js. Auto-instrumentation is initialized before Express. Traces are sent using OTLP/HTTP to a local collector inside the same ECS task.

Listing 8.1: OpenTelemetry initialization principle

```
1 const { NodeSDK } = require('@opentelemetry/sdk-node');
2 const { getNodeAutoInstrumentations } =
3   require('@opentelemetry/auto-instrumentations-node');
4 const { OTLPTraceExporter } =
5   require('@opentelemetry/exporter-trace-otlp-http');
6
```

```

7 const sdk = new NodeSDK({
8   traceExporter: new OTLPTraceExporter({
9     url: process.env.OTEL_EXPORTER_OTLP_TRACES_ENDPOINT
10  }),
11  instrumentations: [getNodeAutoInstrumentations()]
12 });
13
14 sdk.start();

```

The ECS variables used are:

```

1 OTEL_EXPORTER_OTLP_TRACES_ENDPOINT=http://localhost:4318/v1/traces
2 OTEL_SERVICE_NAME=auth-service

```

8.4 ADOT Collector Sidcar

The Auth Task Definition contains a second container:

```

1 Image: public.ecr.aws/aws-observability/aws-otel-collector:latest
2 Port: 4318/TCP
3 Command: --config=/etc/ecs/ecs-default-config.yaml

```

Collector logs confirmed that the OTLP gRPC receiver opened on port 4317 and the OTLP HTTP receiver opened on port 4318, followed by the **ready** state. The Task Role contains X-Ray permissions such as `xray:PutTraceSegments` and `xray:PutTelemetryRecords`.

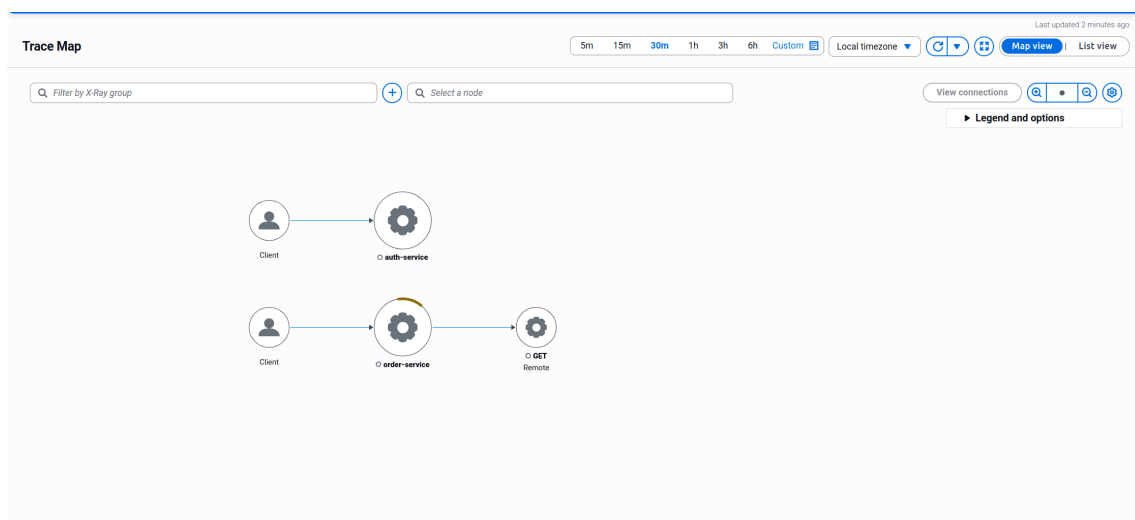


Figure 8.1: X-Ray / CloudWatch Trace Map after generating traffic

Chapter 9

Infrastructure as Code with CloudFormation IaC Generator

9.1 Template Generation

AWS CloudFormation IaC Generator was used to scan existing resources in the region and generate a YAML template representing a significant part of the deployed infrastructure.

The template contains, among other resources:

- VPC, subnets, Internet Gateway, NAT Gateway, and route tables;
- ALB, listener, listener rules, and Target Groups;
- ECS cluster and Task Definitions;
- ECR repositories;
- Secrets Manager;
- IAM roles/policies;
- CodePipeline and the S3 artifact bucket;
- ElastiCache subnet group.

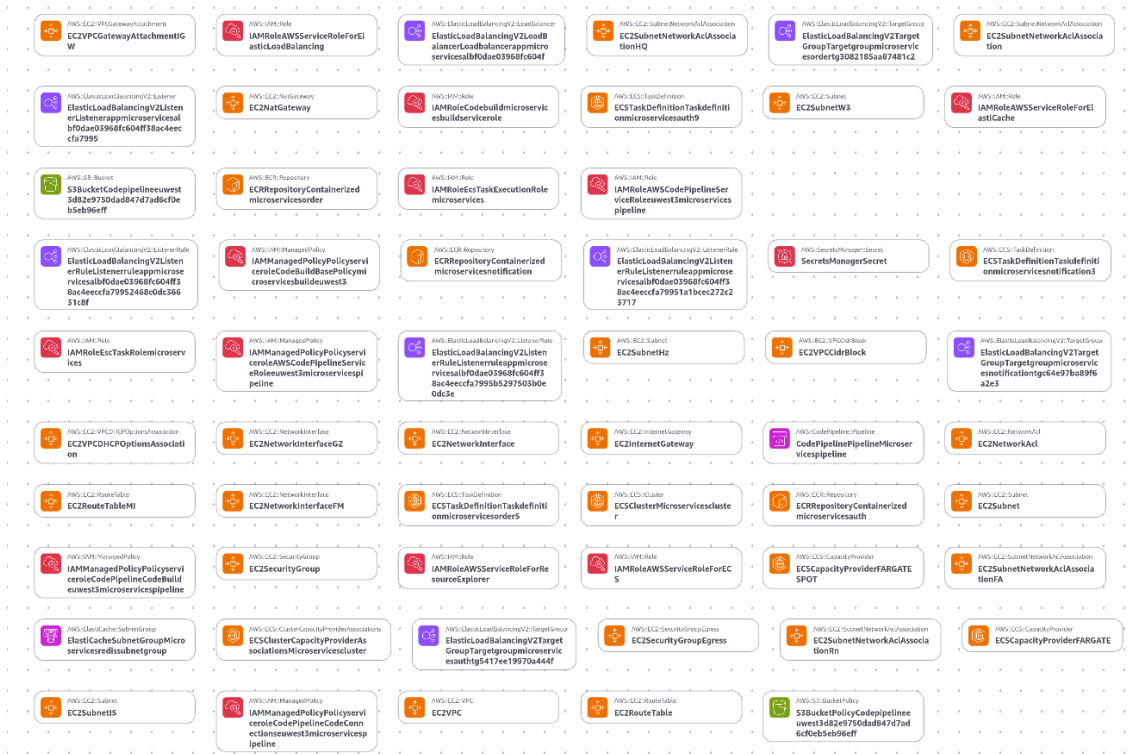


Figure 9.1: CloudFormation IaC Generator after scanning resources

9.2 Limitations of the Generated Template

A template generated from an already-running environment is an excellent starting point, but it should not be treated as a clean template ready to recreate the infrastructure.

Several ephemeral elements were exported:

- ECS network interfaces (ENIs);
- private task IP addresses in Target Groups;
- hardcoded IDs and ARNs;
- many verbose default values.

Conversely, some important resources do not appear completely in the generated snapshot, including ECS Services, the CodeBuild project, some Cloud Map resources, the Redis resource itself, and routing associations required for a fully reproducible deployment.

9.3 Required Cleanup

The next step is to convert this snapshot into real Infrastructure as Code:

- remove ENIs and task IP addresses;
- replace static VPC IDs, Subnet IDs, and ARNs with `!Ref` and `!GetAtt`;
- add missing resources;
- parameterize region, names, and sizes;
- fix JWT secret injection;

- organize the template into networking, compute, security, and CI/CD components.

Result

The work completed therefore enables a gradual transition from a manual deployment built for learning purposes toward versioned and reproducible infrastructure.

Chapter 10

Validation and Results

The main validations completed are:

- all three ECS services can reach the 1/1 **running** state;
- images are built by CodeBuild and pushed to ECR;
- the ALB correctly routes the three API prefixes;
- health checks detect available tasks;
- external calls can be tested with `curl` or with the frontend;
- Cloud Map names enable private service-to-service communication;
- the pipeline is triggered from GitHub and stores its artifacts in S3;
- the ADOT collector runs as a sidecar and exposes the OTLP HTTP receiver on port 4318;
- a base CloudFormation template was generated to initiate IaC work.

Chapter 11

Challenges and Technical Decisions

11.1 The `latest` Tag and Image Versioning

An initial Task Definition pointed to an image tagged `:latest`, while CodeBuild pushed only numeric tags. ECS therefore returned `CannotPullContainerError`. The chosen solution was to explicitly use tags generated by CodeBuild and keep this versioning strategy in the CI/CD pipeline.

11.2 ALB Routing and Application Paths

The ALB selects a Target Group according to the request path, but it does not automatically remove the URL prefix. External routes were therefore standardized around `/api/auth`, `/api/order`, and `/api/notification` so that the API remains consistent with ALB routing rules.

11.3 Local Nginx Configuration

An old Nginx site containing a broken symbolic link named `poste-monitor` prevented the new configuration from loading and caused 500/502 errors. After removing the obsolete entry and validating the configuration with `nginx -t`, the test frontend could be served correctly.

11.4 CodeDeploy Unavailable

The Blue/Green configuration was not completed because CodeDeploy access redirected to the AWS account activation page. The rest of the CI pipeline was nevertheless implemented so that CodeDeploy can be integrated later without changing the overall architecture.

Chapter 12

Future Improvements

The main future improvements are:

1. Complete CodeDeploy Blue/Green deployment and automatic rollback.
2. Add HTTPS using AWS Certificate Manager and a DNS domain.
3. Clean up the CloudFormation template and make the deployment fully reproducible.
4. Enable ECS autoscaling based on CPU, memory, or application metrics.
5. Use Redis for shared data that should not remain in local task memory.
6. Add automated tests to the pipeline before image builds.
7. Add CloudWatch dashboards and alarms for latency, 5xx errors, and service availability.
8. Further tighten IAM policies and scope permissions to specific resources.

Chapter 13

Conclusion

This project made it possible to build a complete Cloud/DevOps pipeline around a microservices application. The resulting architecture combines containerization, private networking, load balancing, dynamic service discovery, secret security, managed caching, CI/CD, and observability.

The main value of the work lies in the integration of these components: each service is relatively simple when considered independently, but orchestrating them on AWS highlights the practical challenges of a distributed platform. The gradual transition toward OpenTelemetry and Cloud-Formation also prepares the project for more advanced Platform Engineering and Infrastructure as Code practices.

Acknowledgments

I would like to thank **Manara** and the program instructors for providing the project framework and practical learning environment that motivated the development of this solution.

Chapter A

Useful Commands

A.1 Local Execution

```
1 git clone https://github.com/salmanebaba/Containerized-Microservices-with-ECS-Fargate-  
   and-Service-Discovery.git  
2 cd Containerized-Microservices-with-ECS-Fargate-and-Service-Discovery  
3 docker compose up --build
```

A.2 Stopping Local Services

```
1 docker compose down
```

A.3 Nginx Tests

```
1 sudo nginx -t  
2 sudo systemctl restart nginx  
3 curl -i http://localhost/  
4 curl -i http://localhost/api/auth/health
```

A.4 Generating Traffic for X-Ray

```
1 for i in {1..20}; do  
2   curl -s http://microservices-alb-176688164.eu-west-3.elb.amazonaws.com/api/auth/  
   health  
3   echo  
4 done
```

Chapter B

X-Ray IAM Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "xray:PutTraceSegments",
8         "xray:PutTelemetryRecords",
9         "xray:GetSamplingRules",
10        "xray:GetSamplingTargets",
11        "xray:GetSamplingStatisticSummaries"
12      ],
13      "Resource": "*"
14    }
15  ]
16 }
```